CLOUD & PLATFORMS

MySQL

From the relational model to production SQL — queries, joins, indexes, normalization, ACID transactions, and connecting MySQL to Python and Node.

mysql · sql · relational-database · indexes · transactions · normalization · query-optimization

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. What MySQL Is and Why It Matters
2. Setup and Getting Started
3. The Relational Model and Data Types
4. DDL: Creating Tables, Keys, and Constraints
5. SQL Queries: SELECT, WHERE, ORDER BY, LIMIT
6. JOINs: Combining Tables
7. Aggregation: GROUP BY, HAVING, and Window Functions
8. Subqueries, CTEs, and Views
9. Indexes and How They Work
10. Normalization: Designing Clean Schemas
11. Transactions and ACID
12. Query Optimization with EXPLAIN
13. A Worked Mini-Example: Monthly Revenue Report
14. Connecting from Python and Node.js
15. Cheat Sheet
16. Best Practices and Common Pitfalls
17. Interview Questions & Answers
18. Further Learning

What MySQL Is and Why It Matters

MySQL is the world's most widely deployed open-source relational database management system (RDBMS). It stores data in **tables** of rows and columns, enforces

structure and relationships, and answers questions about that data using **SQL** — Structured Query Language.

A relational database organizes information into tables that reference one another through **keys**. Instead of duplicating a customer's details inside every order, you store the customer once and point orders at it. This eliminates redundancy, keeps data consistent, and lets you ask complex questions by *joining* tables back together at query time.

RELATIONAL

Data lives in typed tables with declared relationships, not loose documents.

TRANSACTIONAL

The default InnoDB engine gives you ACID guarantees — all-or-nothing changes.

UBIQUITOUS

Powers WordPress, countless SaaS backends, and the LAMP/LEMP stack.



Fig 1. Two tables linked by a foreign key; a JOIN recombines them at query time.

> **Note**
> MySQL and MariaDB share a common ancestry; MariaDB is a community fork created after Oracle's acquisition of MySQL. Syntax overlaps heavily, but this guide targets MySQL 8.x (current LTS line as of 2026, with 8.4 as the long-term support release).

Setup and Getting Started

Install the server and client, start the service, and connect with the `mysql` command-line client.

BASH

```bash
# macOS (Homebrew)
brew install mysql
brew services start mysql

# Debian / Ubuntu
sudo apt update && sudo apt install mysql-server
sudo systemctl start mysql

# Secure the install (sets root password, removes anonymous users)
sudo mysql_secure_installation

# Connect as root
mysql -u root -p

# Connect to a specific host/port/database
mysql -h db.example.com -P 3306 -u app_user -p shop_db
```

Once connected, these meta-commands orient you:

SQL

```sql
SHOW DATABASES;             -- list databases
CREATE DATABASE shop_db;    -- create one
USE shop_db;                -- select it for subsequent commands
SHOW TABLES;                -- list tables in current database
DESCRIBE customers;         -- show a table's columns and types
SHOW CREATE TABLE orders\G -- full DDL; \G formats rows vertically
```

> ✦ **Tip**
> Ending a statement with `\G` instead of `;` prints each row as a vertical key/value block — invaluable for wide tables in the terminal.

The Relational Model and Data Types

A **table** (relation) is a set of **rows** (tuples). Each **column** (attribute) has a fixed data type. Choosing the right type matters for correctness, storage, and index performance.

Category	Types	Use for
Integer	`TINYINT`, `INT`, `BIGINT`	IDs, counts; add `UNSIGNED` when never negative
Decimal	`DECIMAL(p,s)`	Money — exact, no float rounding
Float	`FLOAT`, `DOUBLE`	Scientific/approximate values only
String	`VARCHAR(n)`, `CHAR(n)`, `TEXT`	Names, short text; `TEXT` for large blobs
Date/Time	`DATE`, `DATETIME`, `TIMESTAMP`	`TIMESTAMP` is UTC-aware, range ends 2038
Other	`BOOLEAN`, `ENUM`, `JSON`	Flags, fixed sets, semi-structured data

> ⚠ **Watch out**
> Never store money in `FLOAT` / `DOUBLE`. `0.1 + 0.2 ≠ 0.3` in binary floating point. Use `DECIMAL(10,2)` so arithmetic stays exact.

DDL: Creating Tables, Keys, and Constraints

Data Definition Language (DDL) defines structure: `CREATE`, `ALTER`, `DROP`. Constraints enforce rules the database guarantees even when application code has bugs.

SQL

```sql
CREATE TABLE customers (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(120) NOT NULL,
  email       VARCHAR(255) NOT NULL UNIQUE,       -- no two customers share an email
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE orders (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  customer_id BIGINT UNSIGNED NOT NULL,
  total       DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  status      ENUM('pending','paid','shipped','cancelled') NOT NULL DEFAULT 'pending',
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_orders_customer
    FOREIGN KEY (customer_id) REFERENCES customers(id)
    ON DELETE CASCADE          -- deleting a customer removes their orders
) ENGINE=InnoDB;
```

PRIMARY KEY

Uniquely identifies a row. One per table. In InnoDB it also physically orders the data (clustered index).

FOREIGN KEY

Points at another table's primary key. Enforces referential integrity — you cannot insert an order for a customer that does not exist.

UNIQUE

Guarantees no duplicate values in a column or set of columns (e.g. one account per email).

NOT NULL / DEFAULT

`NOT NULL` forbids missing values; `DEFAULT` supplies one when the client omits the column.

Modifying an existing table:

SQL

```sql
ALTER TABLE customers ADD COLUMN phone VARCHAR(32) NULL;
ALTER TABLE customers MODIFY COLUMN name VARCHAR(200) NOT NULL;
ALTER TABLE customers DROP COLUMN phone;
ALTER TABLE orders ADD INDEX idx_status (status);
```

SQL Queries: SELECT, WHERE, ORDER BY, LIMIT

The `SELECT` statement is how you read data. MySQL evaluates its clauses in a logical order that differs from how you write them — understanding this explains many surprises.

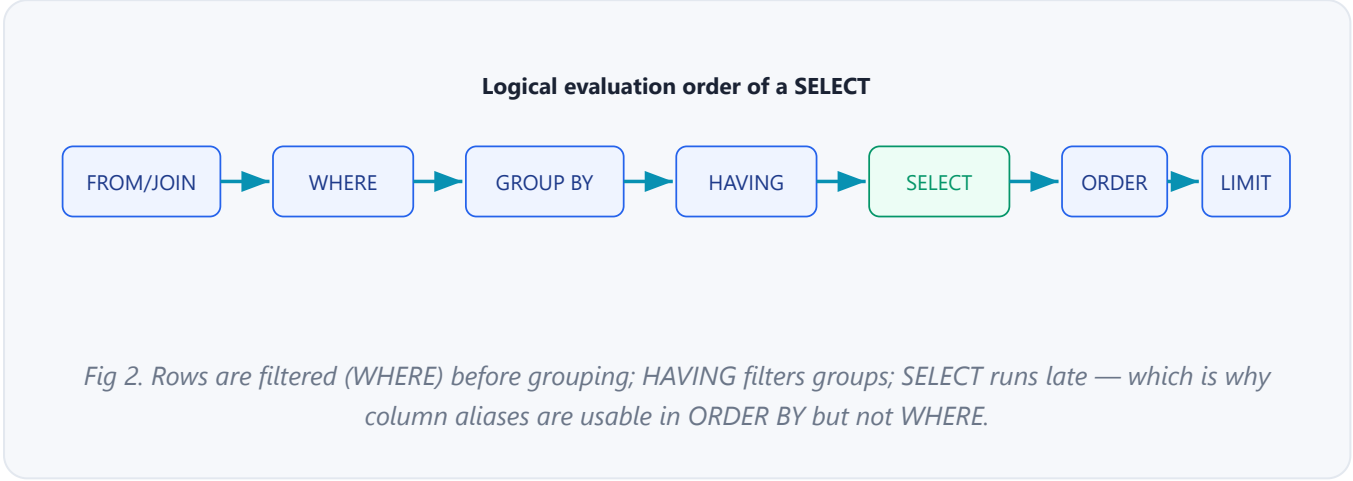


Fig 2. Rows are filtered (WHERE) before grouping; HAVING filters groups; SELECT runs late — which is why column aliases are usable in ORDER BY but not WHERE.

SQL

```sql
SELECT id, name, email
FROM customers
WHERE name LIKE 'A%'          -- starts with A
  AND created_at ≥ '2026-01-01'
ORDER BY created_at DESC
LIMIT 20 OFFSET 0;            -- first page of 20

-- Useful predicates
WHERE status IN ('paid','shipped')
WHERE total BETWEEN 10 AND 100
WHERE phone IS NULL                -- NULL needs IS, never =
WHERE email LIKE '%@gmail.com';
```

⚠ **NULL is not a value**

`NULL = NULL` is `NULL` (unknown), not true. Use `IS NULL` / `IS NOT NULL`. Aggregates like `COUNT(col)` skip NULLs, while `COUNT(*)` counts rows.

JOINs: Combining Tables

A JOIN matches rows across tables on a condition. The join type decides what happens to rows with no match.



MySQL has no native FULL OUTER JOIN — combine LEFT and RIGHT with UNION.

Fig 3. Join types as set operations over two tables.

SQL

```sql
-- INNER: customers who have at least one order
SELECT c.name, o.id AS order_id, o.total
FROM customers c
INNER JOIN orders o ON o.customer_id = c.id;

-- LEFT: every customer, even those with zero orders (NULLs for order columns)
SELECT c.name, COUNT(o.id) AS order_count
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
GROUP BY c.id, c.name;

-- Self join: find customers in the same city
SELECT a.name, b.name
FROM customers a
JOIN customers b ON a.city = b.city AND a.id < b.id;
```

Aggregation: GROUP BY, HAVING, and Window Functions

Aggregate functions collapse many rows into one summary value. `GROUP BY` produces one output row per group; `HAVING` filters those groups (it runs *after* aggregation, unlike `WHERE`).

SQL

```sql
SELECT status,
       COUNT(*)      AS n,
       SUM(total)    AS revenue,
       AVG(total)    AS avg_order,
       MAX(total)    AS biggest
FROM orders
GROUP BY status
HAVING SUM(total) > 1000       -- only groups whose revenue exceeds 1000
ORDER BY revenue DESC;
```

Window functions (MySQL 8.0+) compute across a set of rows *without* collapsing them — perfect for running totals, rankings, and "top N per group".

SQL

```sql
-- Rank each customer's orders by total, keeping every row
SELECT
  customer_id,
  total,
  ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY total DESC) AS rn,
  SUM(total)   OVER (PARTITION BY customer_id)                     AS customer_lifetim
FROM orders;

-- "Latest order per customer" = keep rows where rn = 1 (wrap in a subquery)
```

Function	Returns
`COUNT(*)` / `COUNT(col)`	Row count / non-NULL count
`SUM` / `AVG` / `MIN` / `MAX`	Numeric aggregates
`GROUP_CONCAT(col)`	Concatenated string of group values
`ROW_NUMBER()` / `RANK()` / `DENSE_RANK()`	Ordinal / ranking within a window
`LAG()` / `LEAD()`	Value from a previous / next row

Subqueries, CTEs, and Views

A **subquery** is a query nested inside another. A **CTE** (Common Table Expression, `WITH`) is a named, readable subquery. A **view** is a stored, reusable query that behaves like a virtual table.

SQL

```sql
-- Scalar subquery: customers who spent above the overall average
SELECT c.name
FROM customers c
WHERE (SELECT SUM(total) FROM orders o WHERE o.customer_id = c.id)
      > (SELECT AVG(total) FROM orders);

-- Correlated EXISTS: customers with at least one shipped order
SELECT name FROM customers c
WHERE EXISTS (
  SELECT 1 FROM orders o
  WHERE o.customer_id = c.id AND o.status = 'shipped'
);

-- CTE: cleaner and composable
WITH customer_revenue AS (
  SELECT customer_id, SUM(total) AS revenue
  FROM orders GROUP BY customer_id
)
SELECT c.name, cr.revenue
FROM customer_revenue cr
JOIN customers c ON c.id = cr.customer_id
WHERE cr.revenue > 500;

-- View: freeze the query behind a name
CREATE VIEW v_top_customers AS
SELECT c.id, c.name, SUM(o.total) AS revenue
FROM customers c JOIN orders o ON o.customer_id = c.id
GROUP BY c.id, c.name;

SELECT * FROM v_top_customers WHERE revenue > 1000;
```

> ✦ **Tip**
>
> Prefer `EXISTS` over `IN (SELECT ...)` for correlated existence checks on large tables — it can short-circuit on the first match and handles NULLs more predictably.

Indexes and How They Work

An index is an auxiliary data structure — a **B+tree** in InnoDB — that lets the engine find rows without scanning every one. Without an index, `WHERE email = ?` is a full table scan: O(n). With one, it is a tree descent: O(log n).

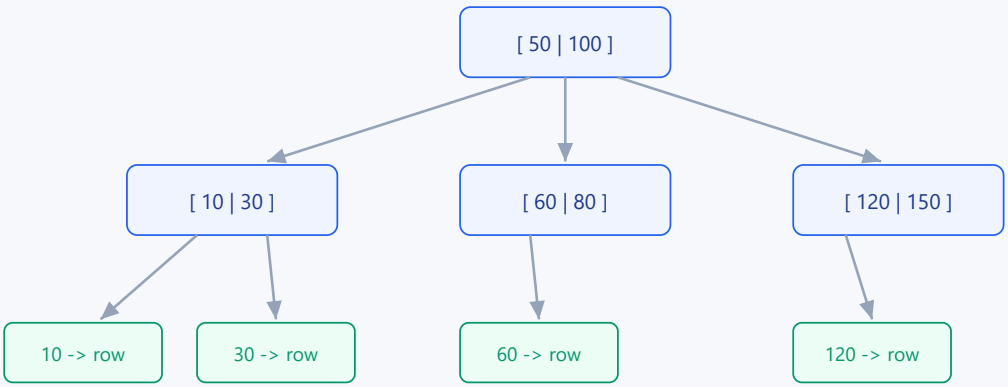


Fig 4. A B+tree index: internal nodes route the search; leaves hold keys pointing to rows.

SQL

```sql
-- Single-column index
CREATE INDEX idx_orders_customer ON orders(customer_id);

-- Composite index: order of columns matters! Follows the "leftmost prefix" rule.
CREATE INDEX idx_orders_cust_status ON orders(customer_id, status);
-- This index helps: WHERE customer_id = 5
--                   WHERE customer_id = 5 AND status = 'paid'
-- It does NOT help:  WHERE status = 'paid'   (skips the leftmost column)

-- Covering index: query answered entirely from the index, no row lookup
CREATE INDEX idx_cover ON orders(customer_id, status, total);
```

WHEN TO INDEX

Columns in `WHERE`, `JOIN ON`, and `ORDER BY`; high-selectivity columns (many distinct values).

COST OF INDEXES

Every `INSERT` / `UPDATE` / `DELETE` must maintain them. They consume disk and RAM. Index deliberately, not reflexively.

⚠ **Index killers**

Wrapping an indexed column in a function or doing a leading wildcard defeats the index: `WHERE YEAR(created_at) = 2026` and `WHERE email LIKE '%@x.com'` both force scans. Rewrite as range predicates (`created_at ≥ '2026-01-01'`).

Normalization: Designing Clean Schemas

Normalization is the process of structuring tables to reduce redundancy and prevent update anomalies. The first three normal forms cover the vast majority of real designs.

Form	Rule	Fixes
1NF	Atomic values; no repeating groups or arrays in a column	Comma-separated lists, repeated `item1,item2` columns
2NF	1NF + every non-key column depends on the *whole* composite key	Partial dependency on part of a key
3NF	2NF + no non-key column depends on another non-key column	Transitive dependencies (e.g. storing `zip` and `city` derivable from zip)

> **Pragmatism**
> Normalize for correctness, then *selectively denormalize* for read performance (e.g. a cached `order_count`) when profiling proves it necessary. Analytics/reporting schemas (star schemas) are deliberately denormalized.

Transactions and ACID

A transaction groups statements into an all-or-nothing unit. InnoDB guarantees **ACID**:

ATOMICITY

All statements commit, or none do. A failure rolls everything back.

CONSISTENCY

Constraints and foreign keys hold before and after the transaction.

ISOLATION

Concurrent transactions don't corrupt each other; the isolation level tunes how much they see.

DURABILITY

Once committed, changes survive a crash (written to the redo log / disk).

SQL

```sql
START TRANSACTION;

UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;

-- If both succeed:
COMMIT;
-- If anything is wrong:
-- ROLLBACK;
```

The **isolation level** controls which concurrency anomalies are possible. MySQL's InnoDB default is `REPEATABLE READ`.

Level	Dirty read	Non-repeatable read	Phantom
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	No	Possible	Possible
REPEATABLE READ default	No	No	Mostly prevented*
SERIALIZABLE	No	No	No

*InnoDB's gap locks largely prevent phantoms at REPEATABLE READ, going beyond the SQL standard.

Deadlocks

Two transactions each holding a lock the other needs will deadlock; InnoDB detects this and kills one with error 1213. Mitigate by acquiring locks in a consistent order and keeping transactions short. Always be ready to retry a deadlocked transaction.

Query Optimization with EXPLAIN

`EXPLAIN` shows the optimizer's execution plan: which indexes it uses, how many rows it estimates, and the access method. It is your single most important tuning tool.

SQL

```sql
EXPLAIN SELECT c.name, SUM(o.total)
FROM customers c JOIN orders o ON o.customer_id = c.id
WHERE o.status = 'paid'
GROUP BY c.id;

-- EXPLAIN ANALYZE actually runs it and reports real timings (MySQL 8.0.18+)
EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id = 42;
```

EXPLAIN column	What to look for
`type`	Access method, best to worst: `const` > `eq_ref` > `ref` > `range` > `index` > `ALL`. `ALL` = full scan, a red flag on big tables.
`key`	The index actually chosen (`NULL` means none).
`rows`	Estimated rows examined — lower is better.
`Extra`	`Using index` (covering, great); `Using filesort` / `Using temporary` (often fixable with an index).



A Worked Mini-Example: Monthly Revenue Report

Suppose a product manager asks: "Show me the top 3 customers by revenue for paid orders in Q1 2026, with their order counts." Here is the full build-up.

Show the complete solution

Connecting from Python and Node.js

Applications talk to MySQL through a driver. The cardinal rule in every language: **use parameterized queries** so user input can never be interpreted as SQL. String-concatenating values is the classic SQL-

injection vulnerability.

Python

BASH

```bash
pip install mysql-connector-python   # official Oracle driver
# or: pip install PyMySQL             # pure-Python alternative
```

PYTHON

```python
import mysql.connector

conn = mysql.connector.connect(
    host="localhost", user="app_user",
    password="secret", database="shop_db",
)
try:
    cur = conn.cursor(dictionary=True)   # rows as dicts

    # Parameterized query - %s placeholders, NEVER f-strings
    cur.execute(
        "SELECT id, name FROM customers WHERE email = %s",
        ("alice@example.com",),
    )
    for row in cur.fetchall():
        print(row["id"], row["name"])

    # A transaction: autocommit is off by default
    cur.execute("UPDATE accounts SET balance = balance - %s WHERE id = %s", (100, 1))
    cur.execute("UPDATE accounts SET balance = balance + %s WHERE id = %s", (100, 2))
    conn.commit()                        # make it durable
except mysql.connector.Error as e:
    conn.rollback()
    raise
finally:
    conn.close()
```

Node.js

BASH

```
npm install mysql2        # supports promises + prepared statements
```

JAVASCRIPT

```
import mysql from 'mysql2/promise';

// A connection pool reuses connections — essential under load.
const pool = mysql.createPool({
  host: 'localhost', user: 'app_user',
  password: 'secret', database: 'shop_db',
  connectionLimit: 10,
});

// Parameterized query with ? placeholders
const [rows] = await pool.execute(
  'SELECT id, name FROM customers WHERE email = ?',
  ['alice@example.com'],
);
console.log(rows);

// Transaction with a dedicated connection
const conn = await pool.getConnection();
try {
  await conn.beginTransaction();
  await conn.execute('UPDATE accounts SET balance = balance - ? WHERE id = ?', [100, 1
  await conn.execute('UPDATE accounts SET balance = balance + ? WHERE id = ?', [100, 2
  await conn.commit();
} catch (err) {
  await conn.rollback();
  throw err;
} finally {
  conn.release();   // return it to the pool
}
```

SQL injection

Never do `" ... WHERE email = '" + userInput + "'"`. One input of `' OR '1'='1` dumps your table. Placeholders (`%s` / `?`) send data separately from SQL text, closing the hole entirely.

Cheat Sheet

Task	Statement
Insert	`INSERT INTO t (a,b) VALUES (1,'x');`
Insert many	`INSERT INTO t (a,b) VALUES (1,'x'),(2,'y');`
Update	`UPDATE t SET b='z' WHERE id=1;`
Delete	`DELETE FROM t WHERE id=1;`
Upsert	`INSERT ... ON DUPLICATE KEY UPDATE b=VALUES(b);`
Distinct	`SELECT DISTINCT city FROM customers;`
Count rows	`SELECT COUNT(*) FROM t;`
Pagination	`... ORDER BY id LIMIT 20 OFFSET 40;`
Conditional	`CASE WHEN total>100 THEN 'big' ELSE 'small' END`
Null default	`COALESCE(phone, 'n/a')`
Add index	`CREATE INDEX ix ON t(col);`
Show plan	`EXPLAIN SELECT ...;`

MySQL vs PostgreSQL vs SQLite (quick comparison)

Aspect	MySQL	PostgreSQL	SQLite
Model	Client/server	Client/server	Embedded file
Strength	Fast reads, ubiquity, replication	Standards, advanced types, extensibility	Zero-config, local apps
JSON	Good	Excellent (JSONB)	Basic
Window fns	8.0+	Yes	3.25+
Typical use	Web backends, WordPress	Complex apps, analytics	Mobile, tests, prototypes

Best Practices and Common Pitfalls

✓ **Do**

Use `InnoDB` (default), declare primary and foreign keys, pick the smallest correct data type, parameterize all queries, use connection pools, and run `EXPLAIN` on any slow query before guessing.

✦ **Do**

Store timestamps in UTC. Set a sensible charset — `utf8mb4` (full Unicode, including emoji) rather than the legacy `utf8` which is only 3 bytes and cannot store all characters.

⚠ **Avoid**

`SELECT *` in application code (fetches unused columns, breaks on schema change), `OFFSET` pagination on huge tables (use keyset/seek pagination), and giant `IN ( ... )` lists.

✕ **Never**

run `UPDATE` / `DELETE` without a `WHERE` clause on production, and never string-concatenate user input into SQL. Enable `sql_safe_updates` in interactive sessions as a guardrail.

Interview Questions & Answers

What is the difference between WHERE and HAVING?

`WHERE` filters individual rows before grouping and cannot reference aggregates. `HAVING` filters groups after `GROUP BY` and can use aggregates like `SUM()`. If there is no `GROUP BY`, put conditions in `WHERE` for efficiency.

Explain the difference between INNER JOIN and LEFT JOIN.

`INNER JOIN` returns only rows with a match in both tables. `LEFT JOIN` returns all rows from the left table and matched rows from the right, filling right-side columns with `NULL` where there is no match — useful for "find records with no related rows" via `WHERE right.id IS NULL`.

What is a primary key versus a unique key?

Both enforce uniqueness. A table has exactly one primary key, which cannot be NULL and (in InnoDB) determines the clustered storage order. A table can have many unique keys, and a unique key column may allow a single NULL (NULLs are treated as distinct).

How does an index speed up queries, and what is the downside?

An index is a sorted B+tree that turns a full table scan (O(n)) into a logarithmic lookup (O(log n)) and enables fast range scans and ordering. The downside: it adds storage and slows writes, since every insert/update/delete must also maintain the index. Over-indexing hurts write-heavy workloads.

What is the leftmost prefix rule for composite indexes?

An index on `(a, b, c)` can be used for predicates on `a`, `a,b`, or `a,b,c` — always starting from the left. A query filtering only on `b` or `c` cannot use it. Design composite indexes with the most selective, most frequently filtered column first.

What are the ACID properties?

Atomicity (all-or-nothing), Consistency (constraints always hold), Isolation (concurrent transactions don't interfere), Durability (committed data survives crashes). InnoDB provides all four; MyISAM does not support transactions.

What isolation anomalies exist and what is MySQL's default?

Dirty reads (seeing uncommitted data), non-repeatable reads (a row changes between reads), and phantom reads (new rows appear in a repeated range query). InnoDB defaults to `REPEATABLE READ`, which prevents dirty and non-repeatable reads and, via gap locks, largely prevents phantoms too.

What is normalization and when would you denormalize?

> Normalization organizes data to eliminate redundancy and update anomalies (1NF atomic values, 2NF no partial key dependency, 3NF no transitive dependency). You denormalize deliberately for read performance — caching an aggregate, or building a star schema for analytics — once profiling shows the join cost is a real bottleneck.

> **How do you prevent SQL injection?**
>
> Use parameterized / prepared statements so values travel separately from the SQL text (`?` or `%s` placeholders), never string concatenation. Add least-privilege database accounts and input validation as defense in depth, but parameterization is the actual fix.

> **What does EXPLAIN tell you, and what is a warning sign?**
>
> It shows the execution plan: access `type`, chosen `key`, estimated `rows`, and `Extra` notes. Red flags are `type: ALL` (full scan) on large tables, `key: NULL` (no index used), and `Using filesort` / `Using temporary` where an index could satisfy the sort or grouping.

> **What is the difference between DELETE, TRUNCATE, and DROP?**
>
> `DELETE` removes rows (optionally filtered), is transactional and logged row-by-row. `TRUNCATE` quickly empties a whole table by recreating it — fast, not row-logged, resets AUTO_INCREMENT, and implicitly commits. `DROP` removes the table definition entirely.

> **What is a covering index?**
>
> An index that contains every column a query needs (in SELECT, WHERE, ORDER BY), so the engine answers entirely from the index without touching the table rows. `EXPLAIN` shows `Using index`. It is one of the most effective single-query optimizations.

Further Learning

- **Official docs**: the MySQL 8.4 Reference Manual — authoritative on syntax, functions, and the InnoDB engine.
- **Practice**: solve joins/aggregation/window problems on sites like LeetCode (Database) and HackerRank (SQL).
- **Depth**: read *High Performance MySQL* for indexing internals, replication, and tuning at scale.
- **Adjacent skills**: database replication and sharding, read replicas, an ORM (SQLAlchemy, Prisma, Sequelize), and migration tools (Flyway, Alembic, Liquibase).
- **Next topics**: query plan caching, partitioning, full-text search, and the JSON column type for semi-structured data.